

Jour 1 – Pandas (fondamentaux)

Lecture, inspection, sélection et nettoyage minimal – objectif 10 minutes par notebook

1. Objectif de la journée et périmètre

Cette journée ne couvre **que** 16 briques : `read_csv`, `head`, `info`, `shape`, `columns`, `loc`, `iloc`, `filter`, `isin`, `isna`, `notna`, `fillna`, `dropna`, `drop_duplicates`, `sort_values`, `value_counts`. Volontairement, **pas de** `groupby`, `merge`, `agg`, `dates` – c’est le programme du Jour 2. Mélanger les deux à ce stade dilue l’entraînement : le but aujourd’hui est que chacune de ces 16 opérations devienne un réflexe pur, tapé sans hésitation, avant d’empiler de la complexité dessus demain.

Pourquoi ce découpage colle au format BCG X. Dans l’épreuve réelle, la Question 1 (« Data analysis ») et la Question 2 (« Data collecting ») commencent toujours de la même façon : charger un ou plusieurs CSV, comprendre leur structure, isoler les bonnes colonnes/lignes, traiter les valeurs manquantes et les doublons. Rien de tout ça ne nécessite un `groupby` ou un `merge` – c’est exactement le socle testé ici.

2. Comment utiliser ce support

1. Faites d’abord le notebook `notebook/day1_pandas_exercices.ipynb` seul, chrono à 10 minutes, sans ouvrir ce PDF ni les solutions.
2. Corrigez avec `solutions/day1_pandas_solutions.ipynb`.
3. Revenez ici pour la partie « **comment raisonner** » de chaque famille de question – c’est ce qui vous fera gagner du temps *le jour de l’épreuve*, pas seulement sur cet exercice précis.
4. Refaites le notebook le lendemain à froid. L’objectif n’est pas d’avoir compris une fois, c’est de taper juste sans réfléchir.

3. Le réflexe de départ : diagnostiquer un fichier inconnu

Avant même de lire l’énoncé en détail, sur n’importe quel CSV nouveau :

```
df = pd.read_csv(path)
df.shape # combien de lignes / colonnes ?
df.head() # a quoi ressemble une ligne ?
df.info() # quels types, combien de non-nuls, quelle memoire ?
df.columns # noms EXACTS a reutiliser (piege frequent : faute de frappe)
df.isna().sum() # ou sont les trous ?
```

COMMENT RAISONNER FACE À CETTE QUESTION

La question BCG ne vous demande jamais explicitement de faire ce scan – mais chaque question suivante (filtrer, nettoyer, agréger) suppose que vous savez déjà à quoi ressemblent les données. Le faire systématiquement en premier, même si l’énoncé ne le demande pas, vous évite de découvrir à mi-parcours qu’une colonne « numérique » est en fait du texte, ou qu’un nom de colonne n’est pas exactement celui que vous supposiez.

4. Inspection – `read_csv`, `head`, `info`, `shape`, `columns`

4.1 Ce que chaque fonction répond

- `pd.read_csv(path)` – charge le fichier. En épreuve, le chemin ou le DataFrame est souvent déjà fourni : ne perdez pas de temps à deviner une arborescence, lisez l’énoncé pour le nom exact du

fichier.

- `df.head(n)` – « à quoi ressemble une ligne ? ». Répond à la question *implicite* : quelle est la granularité du tableau (une ligne = un chauffeur ? une course ? un jour ?).
- `df.shape` – « combien de données ? » Un tuple (`n_lignes`, `n_colonnes`). Sert aussi à vérifier vos résultats intermédiaires (le nombre de lignes a-t-il changé comme prévu après un filtre ?).
- `df.columns` – la liste exacte des noms de colonnes, à copier-coller plutôt qu'à retaper de mémoire.
- `df.info()` – dtype, nombre de non-nuls et mémoire par colonne, en une commande.

SCHÉMA À RECONNAÎTRE

« Nouveau fichier / nouveau DataFrame » → toujours ces cinq lignes avant d'écrire la moindre logique métier.

ERREURS FRÉQUENTES

- Faire confiance au nom de colonne annoncé dans l'énoncé sans vérifier `df.columns` – une majuscule ou un espace en trop fait échouer silencieusement un `df["Rating"]` au lieu de `df["rating"]` (KeyError immédiat, mais fait perdre du temps à comprendre pourquoi).
- Confondre `info()` (dtypes + non-nuls, imprime et renvoie None) et `describe()` (statistiques numériques, renvoie un DataFrame à afficher).

5 5. Sélection – loc, iloc, filter

5.1 La seule règle qui compte

`.loc` raisonne en **étiquettes** (noms de colonnes, valeurs d'index); `.iloc` raisonne en **positions entières** (comme une liste Python). Tout le reste en découle.

```
df.loc[:, ["a", "b"]] # toutes les lignes, colonnes "a" et "b" par nom
df.loc[5:10] # lignes d'étiquette 5 A 10 INCLUS
df.iloc[:10, :3] # 10 premières lignes, 3 premières colonnes (position)
df.iloc[5:10] # lignes de position 5 à 9 (10 EXCLU, comme Python)
df.filter(like="rat") # colonnes dont le NOM contient "rat"
df.filter(items=["a", "b"]) # colonnes listées explicitement
df.filter(regex="^is_") # colonnes dont le nom matche une regex
```

COMMENT RAISONNER FACE À CETTE QUESTION

Pour choisir entre `.loc` et `.iloc`, posez-vous une seule question : « je raisonne en nom de colonne / valeur d'index, ou en position brute (le Nième élément) ? » Dans une épreuve, `.loc` domine largement car les énoncés parlent presque toujours de colonnes nommées. `.iloc` apparaît surtout pour « les N premières lignes » ou un échantillonnage positionnel.

`filter` est à part : il ne filtre **pas des lignes selon une condition sur les valeurs** (erreur de débutant fréquente), il sélectionne des **colonnes (ou lignes) selon leur nom**. Si l'énoncé dit « les colonnes qui commencent par... » ou « qui contiennent le mot ... », c'est `filter`; si l'énoncé dit « les lignes où telle valeur est supérieure à... », c'est un masque booléen (section suivante), pas `filter`.

ERREURS FRÉQUENTES

- `.loc[5:10]` inclut la borne 10; `.iloc[5:10]` ne l'inclut pas. C'est LE piège classique en QCM chronométré.
- Enchaîner `df[condition].iloc[3]` en pensant obtenir la ligne d'étiquette 3 : non, l'index

d'origine n'est pas remis à zéro après un filtre. Pour viser une étiquette précise après filtrage, utilisez `.loc`, ou faites `.reset_index(drop=True)` d'abord.

6 6. Filtrage – masques booléens et `isin`

```
df[df["rating"] > 4.5] # condition simple
df[(df["age"] > 100) | df["age"].isna()] # OR, chaque cond. entre parentheses
df[df["city"].isin(["Paris", "Lyon", "Nice"])] # appartenance a une liste
df[~df["city"].isin(...)] # PAS dans la liste
```

COMMENT RAISONNER FACE À CETTE QUESTION

Dès que l'énoncé combine plusieurs conditions (« et », « ou », « sauf »), la méthode est toujours la même : écrire chaque condition séparément entre parenthèses, puis les combiner avec `&` (et), `|` (ou), `~` (négation) – jamais les mots-clés Python `and/or/not`, qui ne savent pas comparer plusieurs valeurs à la fois et lèvent une `ValueError: truth value is ambiguous`.

Pour « appartient à un ensemble de valeurs », pensez immédiatement à `isin` plutôt qu'une chaîne de `==` reliés par `|` : plus court, plus rapide, et ça passe à l'échelle même avec 50 valeurs à tester.

RACCOURCI PLUS RAPIDE

`isin` bat toujours une chaîne de `(df.c==a) | (df.c==b) | ...` dès qu'il y a plus de 2 valeurs à comparer.

ERREURS FRÉQUENTES

- `df[df.a > 1 and df.b < 2]` → crash. Toujours `&` / `|` sur des masques booléens.
- `isin` compare des valeurs **exactes** : "paris" et "Paris" sont deux valeurs différentes pour `isin`. Normaliser la casse (`.str.lower()`) est un outil de Jour 2 ; aujourd'hui, si les données sont sales, le résultat de `isin` l'est aussi – c'est normal et attendu à ce stade.
- Avec `~isin`, les valeurs NaN de la colonne testée ressortent comme `True` (un NaN n'est jamais « dans » une liste) – vérifiez si l'énoncé veut les manquants inclus ou exclus du résultat « négatif ».

7 7. Valeurs manquantes – `isna`, `notna`, `fillna`, `dropna`

```
df.isna().sum() # nombre de manquants par colonne
(df.isna().mean() * 100).round(1) # pourcentage de manquants par colonne
df[df["rating"].notna()] # lignes ou rating N'EST PAS manquant
s.fillna(s.mean()) # remplacer par une statistique de la colonne
s.fillna("Unknown") # remplacer par une valeur métier explicite
df.dropna(subset=["driver_id"]) # supprimer les lignes ou CETTE colonne manque
```

COMMENT RAISONNER FACE À CETTE QUESTION

Le truc à retenir pour un pourcentage : la moyenne d'un booléen est un taux. `isna()` produit une matrice de `True/False`, c'est-à-dire de 1/0. La **moyenne** d'une colonne booléenne est donc exactement sa **proportion de True**. Ce réflexe (`condition_booléenne.mean() = taux`) revient constamment dans les épreuves BCG X : taux de valeurs manquantes, taux de

succès, part d'une catégorie – toujours la même construction.

Comment choisir la valeur de remplissage. Ce n'est jamais une règle automatique – c'est une question de **sens métier** :

- colonne numérique, distribution à peu près symétrique → la moyenne est défendable ;
- colonne numérique avec des valeurs extrêmes (outliers) → préférer la médiane, moins sensible aux valeurs aberrantes ;
- « absence d'information » qui a un sens métier précis (par exemple « pas encore démarré ») → une valeur fixe justifiée (0), pas une statistique ;
- colonne catégorielle → une catégorie explicite comme "Unknown", jamais une valeur inventée au hasard.

Préparez-vous à **justifier ce choix à l'oral en une phrase** – c'est souvent ce qui distingue une bonne réponse d'une réponse mécanique dans une évaluation BCG X.

dropna(subset=...) plutôt que **dropna()** seul. Sans **subset**, une ligne est supprimée dès qu'une seule colonne quelconque est manquante, même une colonne hors sujet – ça peut vider une table entière sans que vous vous en rendiez compte. Toujours restreindre **subset=** aux colonnes réellement visées par l'énoncé.

ERREURS FRÉQUENTES

- **fillna** ne modifie pas sur place par défaut : il faut réaffecter (**s = s.fillna(...)**), sinon le résultat est perdu.
- **df.fillna(df.mean())** sur **tout** le DataFrame plutôt que colonne par colonne : échoue ou produit des valeurs absurdes sur les colonnes texte.
- Remplir les valeurs manquantes **avant** d'avoir nettoyé les valeurs impossibles (âge négatif, âge de 250 ans) : ces aberrations faussent la moyenne/médiane calculée pour le remplissage. L'ordre correct est : valeur impossible → NaN, puis remplissage.

8. Doublons – drop_duplicates

```
df.duplicated().sum() # combien de doublons EXACTS
df.drop_duplicates() # les supprimer
df.drop_duplicates(subset=["driver_id", keep="last"]) # une ligne par identifiant
assert df["driver_id"].is_unique # toujours VERIFIER
```

COMMENT RAISONNER FACE À CETTE QUESTION

Distinguez toujours **deux** problèmes différents, qui demandent **deux** traitements différents :

- **doublon exact** – toutes les colonnes sont identiques (souvent causé par un double envoi de données) → toujours sûr à supprimer sans réfléchir, aucune perte d'information ;
- **doublon de clé métier** – l'identifiant (**driver_id**) se répète mais le contenu diffère légèrement → il faut **choisir une règle de survivant** explicite (**keep="first"** ou **"last"**), pas se contenter d'un choix par défaut sans le justifier.

Traiter les doublons exacts **avant** les doublons de clé métier : une fois les copies identiques éliminées, le **keep=** de l'étape suivante ne tranche plus qu'entre des lignes **réellement différentes**, ce qui rend le choix significatif.

Une étape de nettoyage sur une clé n'est jamais considérée comme terminée tant qu'elle n'est pas **vérifiée** : **assert df["id"].is_unique** après coup, systématiquement.

SCHEMA À RECONNAÎTRE

« une seule ligne par identifiant » → `drop_duplicates(subset=["id"], keep=...)` puis toujours **vérifier** avec `assert`, jamais supposer que ça a marché.

9 9. Tri et comptage – `sort_values`, `value_counts`

```
df.sort_values("rating", ascending=False) # tri simple
df.sort_values(["city", "age"], ascending=[True, False]) # tri multi-cles
s.value_counts() # effectifs, deja tries decroissant
s.value_counts(dropna=False) # inclut les NaN comme categorie
s.value_counts(normalize=True) * 100 # en pourcentage
s.value_counts().idxmax() # la valeur la plus frequente, en 1 ligne
```

COMMENT RAISONNER FACE À CETTE QUESTION

`sort_values` avec une **liste** de colonnes trie d'abord par la première clé, puis, à égalité, par la seconde – `ascending` accepte alors aussi une liste, un booléen par colonne. Par défaut, les NaN de la colonne triée finissent toujours en dernier, quel que soit le sens du tri.

`value_counts()` répond à « quelle est la distribution de cette colonne catégorielle ? » et trie déjà par fréquence décroissante – inutile d'enchaîner un `sort_values` derrière. Par défaut il **ignore** les NaN : si l'énoncé demande explicitement le décompte « valeurs manquantes incluses », `dropna=False` est obligatoire, sinon le total ne correspondra pas au nombre de lignes du DataFrame.

SCHEMA À RECONNAÎTRE

« classer / top N / pire N » → `sort_values(col, ascending=...)` (+ `.head(N)` au besoin).

« distribution d'une catégorie / valeur la plus fréquente » → `value_counts(dropna=False, normalize=...)`.

10 10. Méthode générale face à une question BCG X

Avant d'écrire la moindre ligne de code, identifiez la **famille** de la question – le réflexe correct en découle automatiquement :

L'énoncé demande...	Famille → réflexe
« combien de lignes/colonnes, quels types »	inspection → <code>shape</code> , <code>info</code> , <code>head</code>
« les colonnes / lignes X à Y »	sélection par position → <code>.iloc</code>
« la colonne nommée ... »	sélection par nom → <code>.loc</code> / <code>filter</code>
« les lignes où telle condition »	masque booléen → <code>df[(cond1) & cond2]</code>
« appartient à une liste de valeurs »	<code>isin</code>
« combien de manquants / quel pourcentage »	<code>isna().sum()</code> / <code>isna().mean()</code>
« remplacer les manquants par... »	<code>fillna</code> (valeur justifiée par colonne)
« supprimer les lignes où ... manque »	<code>dropna(subset=...)</code>
« une seule ligne par identifiant »	<code>drop_duplicates(subset=...)</code> + <code>assert is_unique</code>
« classer / top N »	<code>sort_values(...).head(N)</code>
« distribution / valeur la plus fréquente »	<code>value_counts()</code>
« taux / pourcentage d'une condition »	<code>condition.mean()</code>

COMMENT RAISONNER FACE À CETTE QUESTION

En épreuve chronométrée, l'erreur la plus coûteuse n'est pas de ne pas connaître la syntaxe – c'est de passer 3 minutes à **ré-dériver** une solution que vous connaissez déjà parce que vous n'avez pas reconnu la famille de la question en 5 secondes. L'objectif de cette table n'est pas de la relire pendant l'épreuve : c'est de l'avoir intériorisée au point de sauter directement l'étape « reconnaissance ».

11 11. Check-list finale

- ☐ le scan de 5 lignes (`shape/head/info/columns/isna`) fait sans réfléchir
- ☐ différence `.loc` (étiquettes, inclusif) vs `.iloc` (positions, exclusif)
- ☐ `filter` = sélection de colonnes par nom, pas un filtre de lignes
- ☐ masque booléen multi-conditions avec `&/|/~` et parenthèses
- ☐ `isin` pour « appartient à une liste », `~isin` pour l'inverse
- ☐ taux/pourcentage via la moyenne d'un booléen (`isna().mean()`, etc.)
- ☐ `fillna` avec une valeur choisie et justifiée par colonne, jamais globale
- ☐ `dropna(subset=...)` ciblé, jamais `dropna()` nu par réflexe
- ☐ doublons exacts **puis** doublons de clé métier, avec `assert is_unique`
- ☐ `sort_values` multi-clés avec listes `by/ascending`
- ☐ `value_counts(dropna=False, normalize=...)` et `idxmax()`
- ☐ le notebook `day1_pandas_exercices.ipynb` bouclé en moins de 10 minutes